
One list, two names

Sample lesson 001 | 2 October 2026 | Read: 5 minutes | Practise: 5 minutes

Objective: predict when changing one Python list also changes what another variable shows.

Demo context: a hypothetical beginner who knows variables and can run simple Python. This sample tests the written-source fallback: no Supadata connection was available, so the lesson uses official Python documentation. No YouTube transcript was retrieved and no schedule was created.

The idea

A list holds a sequence of items. A variable can refer to that list. Assigning the variable to a second name does not create another list: both names refer to the same object. Changing that object through either name is visible through the other. The Python tutorial explicitly explains this behavior. [1]

Imagine drafting a shopping list and calling a second variable your backup. A second name alone does not preserve an earlier version. This is a useful distinction whenever you prepare a draft, test a change or retain an original collection.

Predict before running

```
shopping = ["rice", "beans"]
backup = shopping
shopping.append("tea")
print(backup)
```

The output is `['rice', 'beans', 'tea']`. The name `backup` sounds like a separate copy, but Python follows the assignment, not the name's English meaning. Both variables still refer to one list.

Make a separate outer list

Use `copy()` when you want a new list. The documented method makes a shallow copy: a new outer list containing references to the same items. `append()` adds an item to the end of the list on which it is called. [2]

```
shopping = ["rice", "beans"]
backup = shopping.copy()
shopping.append("tea")
print(backup)
```

Now the output is `['rice', 'beans']`. Appending to `shopping` changes its outer list, while `backup` refers to a different outer list. These are original teaching examples; both outputs were checked by executing the code.

Practice

Use a Python interpreter you already have. If you cannot run Python, first predict the answer on paper; execution can wait until a suitable tool is available.

```
tasks = ["email", "invoice"]
tomorrow = tasks
tomorrow.append("call")
print(tasks)
```

1. Predict the printed list before running the code.
2. Run it and compare your prediction with the output.
3. Change only the second line so adding a task to tomorrow leaves tasks unchanged.
4. Explain what changed without saying simply, "copy fixes it."

Completion check: you can produce ['email', 'invoice'] from the changed version and explain why there are now two outer lists.

Watch for

A shallow copy does not independently duplicate nested mutable objects. If your list contains other lists, those inner lists can still be shared. Python's copy documentation distinguishes this from deep copying. [3] For this exercise, use the flat lists of strings shown above; nested collections can be a later lesson.

Avoid learning a rule that says to copy every list. The decision is whether your task needs shared changes or a separate collection.

Check your understanding

1. Why did backup contain tea in the first example?
2. Which line would you change in the practice exercise, and what would you replace it with?

Answers

1. Both names referred to the same list, and that list was changed.
2. Replace the second line with `tomorrow = tasks.copy()`. Appending now changes the new outer list.

Sources and evidence

[1] [Python tutorial: Lists](#). Selected because it explains assignment and list mutation directly.

[2] [Python tutorial: List methods](#). Checked the definitions of `append()` and `copy()`.

[3] [Python library reference: copy](#). Checked the shallow-copy limitation.

All three are Python Software Foundation documentation, accessed 2 October 2026. Relevant sections were read; this is an original explanation with independently executed examples. No claims about video popularity or comments are made.

Next skill: removing an item without accidentally modifying a list you still need. Learner comprehension and exercise completion remain unknown.

Optional feedback: was predicting the result or changing the code harder?